

# 在当前 agent\_service 中实现上下文系统

Redis DB 4 热上下文 + MongoDB 持久化 + LangGraph 加载、压缩、保存

基于当前仓库代码编写 · 2026-08-28 · 只讲项目如何增加和修改

### 这次只完成一个闭环

当前请求先从 Redis/MongoDB 加载模型上下文；过长时压缩；模型回答校验成功后再保存。MySQL 展示消息和请求 history 仍然不参与模型上下文。

## 先看最终运行流程

改造后的 LangGraph

```
input_guard
→ load_context
→ compact_context
→ generate
→ output_validate
→ persist_context
```

你当前的最小图已经有 input\_guard、generate 和 output\_validate。我们不推倒重写，只在中间插入 load\_context、compact\_context，在末尾插入 persist\_context。SSE、取消注册表、角色文件和 Java DTO 保持原样。

| 操作 | 当前项目文件                                | 本步结果                 |
|----|---------------------------------------|----------------------|
| 修改 | pyproject.toml、.env.example、config.py | 增加 Redis/Mongo/预算配置  |
| 新增 | schemas/context.py                    | 定义模型上下文快照            |
| 新增 | services/context_repository.py        | Redis 优先、Mongo 回源和保存 |

| 操作    | 当前项目文件                            | 本步结果             |
|-------|-----------------------------------|------------------|
| 新增    | services/context_manager.py       | 装配、估算、压缩和追加轮次    |
| 修改    | models/gateway.py                 | 增加非流式摘要能力        |
| 修改    | graph/state.py、graph/workflow.py  | 把上下文接进 LangGraph |
| 修改    | services/agent_runtime.py、main.py | 依赖注入和客户端生命周期     |
| 修改/新增 | api/routes/chat.py、tests          | 错误映射和离线测试        |

## 第 1 步：增加依赖和环境变量

先只做连接层准备，不修改 LangGraph。这样如果依赖或配置有问题，可以在改动业务流程前定位。

**执行命令** agent\_service/pyproject.toml + uv.lock

```
# 在 agent_service 根目录执行。uv 会同时更新 pyproject.toml 和 uv.lock。
uv add "redis>=6.4,<9" "pymongo>=4.14,<5"

# 安装完成后先确认依赖没有破坏原有项目。
uv run ruff check .
uv run pytest
```

**追加配置** agent\_service/.env.example 和你本地的 .env

```
# Redis: 继续使用现有实例，但 Agent 固定连接 database 4。
AGENT_REDIS_HOST=192.168.100.128
AGENT_REDIS_PORT=6379
AGENT_REDIS_PASSWORD=<你的 Redis 密码>
AGENT_REDIS_DATABASE=4
AGENT_REDIS_CONNECT_TIMEOUT_SECONDS=3
AGENT_REDIS_SOCKET_TIMEOUT_SECONDS=3
AGENT_CONTEXT_REDIS_TTL_SECONDS=604800

# MongoDB: 如果使用 docker run 创建的 mongo/mongo root 账号，authSource 必须是 admin。
# 后续若创建 agent_context_app 应用账号，再把用户名、密码和 authSource 换成目标数据库。
AGENT_MONGODB_URI=mongodb://mongo:mongo@192.168.100.128:27017/xinchuang_agent_context?authSource=admin
AGENT_MONGODB_DATABASE=xinchuang_agent_context
AGENT_MONGODB_SERVER_SELECTION_TIMEOUT_MS=3000
```

```
AGENT_MONGODB_CONNECT_TIMEOUT_MS=3000
```

```
# 上下文预算：第一版先使用近似字符计数，后续可替换为模型 tokenizer。
```

```
AGENT_CONTEXT_SOFT_TOKEN_LIMIT=8000
AGENT_CONTEXT_HARD_TOKEN_LIMIT=12000
AGENT_CONTEXT_RECENT_TURNS=6
AGENT_CONTEXT_SUMMARY_MAX_TOKENS=1200
AGENT_CONTEXT_CHARS_PER_TOKEN=2.0
```

## MongoDB 数据库不需要预先手工创建

配置项本身不会创建数据库；第一次 createCollection、创建索引或写入文档时，MongoDB 才会真正创建 xinchuang\_agent\_context。你当前使用 docker run 创建的 root 用户 mongo/mongo，因此 authSource=admin。

### 修改文件 agent\_service/src/agent\_service/config.py

把字段放进现有 Settings 类，并把最后的预算校验合并进现有 model\_validator。因为项目已经使用 AGENT\_ 前缀，所以 redis\_host 会自动读取 AGENT\_REDIS\_HOST。

```
# 文件: src/agent_service/config.py
# 操作: 把下面字段加入现有 Settings 类中。

# Redis 连接配置。SecretStr 可以避免调试输出时直接显示密码。
redis_host: str = "192.168.100.128"
redis_port: int = Field(default=6379, ge=1, le=65535)
redis_password: SecretStr | None = None
redis_database: int = Field(default=4, ge=0)
redis_connect_timeout_seconds: float = Field(default=3.0, gt=0)
redis_socket_timeout_seconds: float = Field(default=3.0, gt=0)
context_redis_ttl_seconds: int = Field(default=604800, ge=60)

# MongoDB 是持久化真相源，因此 URI 必须由环境变量提供。
mongodb_uri: SecretStr
mongodb_database: str = "xinchuang_agent_context"
mongodb_server_selection_timeout_ms: int = Field(default=3000, gt=0)
mongodb_connect_timeout_ms: int = Field(default=3000, gt=0)

# 上下文压缩和模型输入预算。
context_soft_token_limit: int = Field(default=8000, gt=0)
context_hard_token_limit: int = Field(default=12000, gt=0)
context_recent_turns: int = Field(default=6, ge=1)
context_summary_max_tokens: int = Field(default=1200, ge=128)
context_chars_per_token: float = Field(default=2.0, gt=0)

# 把这段校验合并到现有 validate_model_credentials 方法的末尾。
if self.context_soft_token_limit >= self.context_hard_token_limit:
    raise ValueError(
        "AGENT_CONTEXT_SOFT_TOKEN_LIMIT 必须小于 "
        "AGENT_CONTEXT_HARD_TOKEN_LIMIT"
```

)

## 第 2 步：定义上下文快照

这一层只负责数据形状，不连接数据库。先把“模型真正需要记住什么”固定下来，后面 Redis、MongoDB 和 LangGraph 都使用同一个模型。

**新增文件** agent\_service/src/agent\_service/schemas/context.py

```
"""模型上下文的数据结构。

这里的数据专门给模型使用，不是 MySQL chat_message 的镜像。
"""

from datetime import UTC, datetime

from pydantic import BaseModel, ConfigDict, Field
from pydantic.alias_generators import to_camel

class ContextModel(BaseModel):
    """统一使用 Python snake_case、Redis/MongoDB camelCase。"""

    model_config = ConfigDict(
        alias_generator=to_camel,
        populate_by_name=True,
        extra="forbid",
    )

class ContextEntry(ContextModel):
    """一轮已经完成的模型上下文。

    user_text 和 assistant_text 可以经过规范化或压缩，不要求与 MySQL
    最终展示文本逐字相同。
    """

    request_id: int = Field(gt=0)
    user_text: str
    assistant_text: str
    created_at: datetime = Field(default_factory=lambda: datetime.now(UTC))

class ContextSnapshot(ContextModel):
    """一个会话当前可供模型使用的完整上下文快照。"""

    schema_version: int = 1
    user_id: int = Field(gt=0)
    session_id: int = Field(gt=0)
```

```

revision: int = Field(default=0, ge=0)
summary: str = ""
facts: list[str] = Field(default_factory=list)
recent_entries: list[ContextEntry] = Field(default_factory=list)
estimated_tokens: int = Field(default=0, ge=0)
last_request_id: int | None = None
updated_at: datetime = Field(default_factory=lambda: datetime.now(UTC))

@classmethod
def empty(cls, *, user_id: int, session_id: int) -> "ContextSnapshot":
    """首次会话没有任何上下文时, 返回 revision=0 的空快照。"""

    return cls(user_id=user_id, session_id=session_id)

```

### 为什么不保存 role/content 列表?

快照同时需要摘要、事实、revision、Token 估算和 requestId 等信息。直接保存聊天消息数组，后面很难安全压缩，也容易误把 MySQL 展示消息当成模型上下文。

## 第 3 步：增加上下文异常

先定义领域异常，Repository 和 ContextManager 只负责抛异常，FastAPI 路由最后统一转换为 SSE error。

**修改文件** agent\_service/src/agent\_service/core/exceptions.py

```

# 文件: src/agent_service/core/exceptions.py
# 操作: 在现有异常类后面增加以下三个异常。

class ContextStoreUnavailableError(AgentServiceError):
    """MongoDB 无法完成必要读取或写入, 本轮不能继续。"""

class ContextRevisionConflictError(AgentServiceError):
    """同一会话被并发更新, 当前快照 revision 已经过期。"""

class ContextTooLargeError(AgentServiceError):
    """压缩和裁剪后仍然超过模型输入硬预算。"""

```

## 第 4 步：实现 Redis + MongoDB Repository

Repository 是整个上下文系统的存储边界。LangGraph 不应该直接拼 Redis Key 或 Mongo 查询，否则节点会越来越难测试。

**新增文件** agent\_service/src/agent\_service/services/context\_repository.py

```
"""Redis + MongoDB 上下文仓储。

读取时 Redis 优先、MongoDB 回源；写入时 MongoDB 优先、Redis 尽力刷新。
"""

import logging
from datetime import UTC, datetime
from typing import Protocol

from pydantic import ValidationError
from pymongo import AsyncMongoClient
from pymongo.errors import DuplicateKeyError, PyMongoError
from redis.asyncio import Redis
from redis.exceptions import RedisError

from agent_service.core.exceptions import (
    ContextRevisionConflictError,
    ContextStoreUnavailableError,
)
from agent_service.schemas.context import ContextSnapshot

logger = logging.getLogger(__name__)

class ContextRepository(Protocol):
    """LangGraph 只依赖这个协议，测试可以换成内存实现。"""

    async def load(self, *, user_id: int, session_id: int) -> ContextSnapshot:
        """加载指定用户会话的上下文。"""

    async def save(
        self,
        snapshot: ContextSnapshot,
        *,
        expected_revision: int,
    ) -> ContextSnapshot:
        """按预期 revision 保存，并返回持久化后的新快照。"""

class RedisMongoContextRepository:
    """生产环境使用的双层仓储。"""

    def __init__(
        self,
        *,
        redis_client: Redis,
        mongo_client: AsyncMongoClient,
```

```

        mongo_database: str,
        redis_ttl_seconds: int,
    ) -> None:
        self._redis = redis_client
        database = mongo_client[mongo_database]
        self._sessions = database["context_sessions"]
        self._turns = database["context_turns"]
        self._redis_ttl_seconds = redis_ttl_seconds

    @staticmethod
    def _key(user_id: int, session_id: int) -> str:
        """Key 同时包含版本、用户和会话，避免不同结构互相覆盖。"""

        return f"xc:agent:context:v1:session:{user_id}:{session_id}"

    async def load(self, *, user_id: int, session_id: int) -> ContextSnapshot:
        key = self._key(user_id, session_id)

        # 1. Redis 是热缓存。命中后刷新 TTL，实现七天滑动过期。
        try:
            cached = await self._redis.get(key)
            if cached:
                snapshot = ContextSnapshot.model_validate_json(cached)
                await self._redis.expire(key, self._redis_ttl_seconds)
                return snapshot
        except (RedisError, ValidationError, ValueError):
            # 缓存故障不能阻断聊天；MongoDB 才是持久化真相源。
            logger.warning(
                "Redis 上下文读取失败，改从 MongoDB 回源，userId=%s sessionId=%s",
                user_id,
                session_id,
                exc_info=True,
            )

        # 2. Redis 未命中或不可用时读取 MongoDB。
        try:
            document = await self._sessions.find_one(
                {"userId": user_id, "sessionId": session_id}
            )
        except PyMongoError as exc:
            raise ContextStoreUnavailableError("MongoDB 上下文读取失败") from exc

        if document is None:
            return ContextSnapshot.empty(user_id=user_id, session_id=session_id)

        document.pop("_id", None)
        snapshot = ContextSnapshot.model_validate(document)

        # 3. 回源成功后尽力回填 Redis。失败只记日志，不影响返回。
        await self._write_cache(snapshot)
        return snapshot

    async def save(
        self,
        snapshot: ContextSnapshot,
        *,
        expected_revision: int,
    ) -> ContextSnapshot:
        """先保存 MongoDB，再刷新 Redis。"""

```

```

lastRequestId 用于重复请求幂等; revision 用于同一会话并发保护。
"""

if snapshot.last_request_id is None:
    raise ValueError("保存已完成上下文时必须提供 last_request_id")

identity = {
    "userId": snapshot.user_id,
    "sessionId": snapshot.session_id,
}

try:
    # 请求重试时, 如果 MongoDB 已经保存过同一个 requestId, 就直接返回。
    current_document = await self._sessions.find_one(identity)
    if (
        current_document is not None
        and current_document.get("lastRequestId") == snapshot.last_request_id
    ):
        current_document.pop("_id", None)
        current = ContextSnapshot.model_validate(current_document)
        await self._ensure_turn(current)
        await self._write_cache(current)
        return current

    saved = snapshot.model_copy(
        update={
            "revision": expected_revision + 1,
            "updated_at": datetime.now(UTC),
        },
        deep=True,
    )
    mongo_document = saved.model_dump(mode="python", by_alias=True)

    # expected_revision=0 时允许首次 upsert; 后续必须命中旧 revision。
    result = await self._sessions.replace_one(
        {**identity, "revision": expected_revision},
        mongo_document,
        upsert=expected_revision == 0,
    )
    if result.matched_count == 0 and result.upserted_id is None:
        raise ContextRevisionConflictError("上下文 revision 已经过期")

    # turn 使用 requestId 唯一索引; 重复执行不会插入第二条。
    await self._ensure_turn(saved)
except ContextRevisionConflictError:
    raise
except DuplicateKeyError as exc:
    # 常见原因是两个请求同时从 revision=0 创建同一个会话。
    raise ContextRevisionConflictError("上下文并发创建冲突") from exc
except PyMongoError as exc:
    raise ContextStoreUnavailableError("MongoDB 上下文保存失败") from exc

# MongoDB 已经成功, Redis 失败只会造成下一轮回源, 不回滚本轮。
await self._write_cache(saved)
return saved

async def _ensure_turn(self, snapshot: ContextSnapshot) -> None:
    """把本轮上下文以 requestId 幂等保存到 context_turns。"""

```



```

        if not snapshot.recent_entries:
            return
        entry = snapshot.recent_entries[-1]
        document = {
            "requestId": entry.request_id,
            "userId": snapshot.user_id,
            "sessionId": snapshot.session_id,
            "userContext": entry.user_text,
            "assistantContext": entry.assistant_text,
            "createdAt": entry.created_at,
        }
        await self._turns.update_one(
            {"requestId": entry.request_id},
            {"$setOnInsert": document},
            upsert=True,
        )

    async def _write_cache(self, snapshot: ContextSnapshot) -> None:
        """尽力写 Redis, 并设置七天 TTL。"""

        try:
            await self._redis.set(
                self._key(snapshot.user_id, snapshot.session_id),
                snapshot.model_dump_json(by_alias=True),
                ex=self._redis_ttl_seconds,
            )
        except RedisError:
            logger.warning(
                "Redis 上下文写入失败, 后续请求将从 MongoDB 回源",
                exc_info=True,
            )

class InMemoryContextRepository:
    """测试专用仓储, 避免 pytest 连接开发者的 Redis/MongoDB。"""

    def __init__(self) -> None:
        self._items: dict[tuple[int, int], ContextSnapshot] = {}

    async def load(self, *, user_id: int, session_id: int) -> ContextSnapshot:
        snapshot = self._items.get((user_id, session_id))
        if snapshot is None:
            return ContextSnapshot.empty(user_id=user_id, session_id=session_id)
        return snapshot.model_copy(deep=True)

    async def save(
        self,
        snapshot: ContextSnapshot,
        *,
        expected_revision: int,
    ) -> ContextSnapshot:
        current = await self.load(
            user_id=snapshot.user_id,
            session_id=snapshot.session_id,
        )
        if current.last_request_id == snapshot.last_request_id:
            return current
        if current.revision != expected_revision:
            raise ContextRevisionConflictError("测试仓储 revision 冲突")
        saved = snapshot.model_copy(
            update={"revision": expected_revision + 1},
            deep=True,
        )

```

```
self._items[(saved.user_id, saved.session_id)] = saved
return saved.model_copy(deep=True)
```

### 先 MongoDB，后 Redis

MongoDB 写入失败意味着完整上下文没有可靠保存，本轮应返回错误；Redis 写入失败只会让下一轮多一次 MongoDB 回源，因此记录告警即可。

## 第 5 步：给模型 Gateway 增加摘要能力

你当前的 ModelGateway 只有 stream，适合最终回答；上下文压缩更适合一次返回短摘要，因此增加 complete。Mock 和真实 Gateway 都要实现，Protocol 才不会失配。

**修改文件** agent\_service/src/agent\_service/models/gateway.py

```
# 文件: src/agent_service/models/gateway.py
# 操作 1: 在 ModelGateway Protocol 中增加非流式 complete 能力。

async def complete(
    self,
    messages: Sequence[BaseMessage],
    *,
    max_output_tokens: int,
) -> str:
    """一次性返回短文本，主要供上下文压缩总结使用。"""

# 操作 2: 在 MockModelGateway 中增加实现，保证离线测试可运行。
async def complete(
    self,
    messages: Sequence[BaseMessage],
    *,
    max_output_tokens: int,
) -> str:
    """Mock 只生成确定性摘要，不访问外部模型。"""

    del messages, max_output_tokens
    return "此前会话包含用户问题、已执行的排查步骤和仍需继续处理的事项。"

# 操作 3: 在 OpenAIModelGateway 中增加真实模型实现。
async def complete(
    self,
    messages: Sequence[BaseMessage],
    *,
    max_output_tokens: int,
) -> str:
```

```
"""使用同一个 LangChain ChatOpenAI 实例完成短摘要。"""
```

```
response = await self._model.ainvoke(
    messages,
    max_tokens=max_output_tokens,
)
return _extract_text(response.content).strip()
```

## 第 6 步：实现装配、Token 预算和压缩

ContextManager 不负责数据库，只负责把快照变成模型消息。这样可以直接用内存对象测试压缩，不需要启动 Redis 或 MongoDB。

**新增文件** agent\_service/src/agent\_service/services/context\_manager.py

```
"""上下文装配、Token 估算和压缩逻辑。"""
```

```
import logging
from math import ceil
```

```
from langchain_core.messages import AIMessage, BaseMessage, HumanMessage, SystemMessage
```

```
from agent_service.config import Settings
from agent_service.core.exceptions import ContextTooLargeError
from agent_service.models.gateway import ModelGateway
from agent_service.schemas.context import ContextEntry, ContextSnapshot
```

```
logger = logging.getLogger(__name__)
```

```
class ContextManager:
```

```
    """把仓储快照转换为模型消息，并在需要时压缩旧上下文。"""
```

```
    def __init__(self, settings: Settings) -> None:
        self._soft_limit = settings.context_soft_token_limit
        self._hard_limit = settings.context_hard_token_limit
        self._recent_turns = settings.context_recent_turns
        self._summary_max_tokens = settings.context_summary_max_tokens
        self._chars_per_token = settings.context_chars_per_token
```

```
    def estimate_text_tokens(self, text: str) -> int:
        """第一版使用字符数近似 Token，中文按 2 字符约 1 Token 估算。"""

        return ceil(len(text) / self._chars_per_token)
```

```
    def needs_compaction(
        self,
        *,
        snapshot: ContextSnapshot,
        system_prompt: str,
        current_message: str,
```

```

) -> bool:
    messages = self.build_messages(
        snapshot=snapshot,
        system_prompt=system_prompt,
        current_message=current_message,
    )
    return self._estimate_messages(messages) > self._soft_limit

async def prepare(
    self,
    *,
    snapshot: ContextSnapshot,
    system_prompt: str,
    current_message: str,
    model_gateway: ModelGateway,
) -> tuple[ContextSnapshot, list[BaseMessage]]:
    """必要时压缩旧轮次，然后返回本轮真正传给模型的消息。"""

    working = snapshot.model_copy(deep=True)
    messages = self.build_messages(
        snapshot=working,
        system_prompt=system_prompt,
        current_message=current_message,
    )
    if self._estimate_messages(messages) <= self._soft_limit:
        return working, messages

    # 只保留最近 N 轮原文，更旧的轮次交给摘要模型。
    recent = working.recent_entries[-self._recent_turns :]
    older = working.recent_entries[: -self._recent_turns]
    if older:
        try:
            summary = await model_gateway.complete(
                self._summary_messages(working.summary, working.facts, older),
                max_output_tokens=self._summary_max_tokens,
            )
            if not summary:
                raise ValueError("摘要模型返回空内容")
        except Exception:
            # 压缩模型失败时采用确定性兜底，不能直接把全部旧轮次丢掉。
            logger.warning("上下文摘要模型失败，使用确定性摘要兜底", exc_info=True)
            summary = self._fallback_summary(working.summary, older)

        working = working.model_copy(
            update={"summary": summary, "recent_entries": recent},
            deep=True,
        )

    messages = self.build_messages(
        snapshot=working,
        system_prompt=system_prompt,
        current_message=current_message,
    )

    # 摘要后仍超过硬预算时，从最旧的近期轮次开始裁剪。
    while (
        self._estimate_messages(messages) > self._hard_limit
        and len(working.recent_entries) > 1
    ):
        working.recent_entries.pop(0)
        messages = self.build_messages(
            snapshot=working,

```

```

        system_prompt=system_prompt,
        current_message=current_message,
    )

    if self._estimate_messages(messages) > self._hard_limit:
        raise ContextTooLargeError("当前问题和必要上下文超过模型输入硬预算")

    return working, messages

def build_messages(
    self,
    *,
    snapshot: ContextSnapshot,
    system_prompt: str,
    current_message: str,
) -> list[BaseMessage]:
    """按固定顺序装配模型消息；绝不读取请求里的 history。"""

    messages: list[BaseMessage] = [SystemMessage(content=system_prompt)]
    if snapshot.summary:
        messages.append(
            SystemMessage(content=f"此前会话摘要: \n{snapshot.summary}")
        )
    if snapshot.facts:
        facts = "\n".join(f"- {fact}" for fact in snapshot.facts)
        messages.append(SystemMessage(content=f"已确认事实: \n{facts}"))
    for entry in snapshot.recent_entries:
        messages.append(HumanMessage(content=entry.user_text))
        messages.append(AIMessage(content=entry.assistant_text))
    messages.append(HumanMessage(content=current_message))
    return messages

def append_completed_turn(
    self,
    *,
    snapshot: ContextSnapshot,
    request_id: int,
    user_text: str,
    assistant_text: str,
) -> ContextSnapshot:
    """仅在回答校验成功后，把完整一轮加入待保存快照。"""

    entry = ContextEntry(
        request_id=request_id,
        user_text=user_text,
        assistant_text=assistant_text,
    )
    entries = [*snapshot.recent_entries, entry]
    estimated = self.estimate_text_tokens(
        snapshot.summary
        + "\n".join(snapshot.facts)
        + "\n".join(
            item.user_text + item.assistant_text for item in entries
        )
    )
    return snapshot.model_copy(
        update={
            "recent_entries": entries,
            "estimated_tokens": estimated,
            "last_request_id": request_id,
        },
        deep=True,
    )

```

```

def _estimate_messages(self, messages: list[BaseMessage]) -> int:
    return sum(self.estimate_text_tokens(str(item.content)) for item in messages)

@staticmethod
def _summary_messages(
    old_summary: str,
    facts: list[str],
    older: list[ContextEntry],
) -> list[BaseMessage]:
    material = "\n\n".join(
        f"用户: {item.user_text}\n 助手: {item.assistant_text}" for item in older
    )
    prompt = (
        "请压缩以下客服上下文。保留用户目标、明确约束、已确认事实、"
        "已完成步骤和未解决事项；删除寒暄、重复描述和无关细节。 \n\n"
        f"已有摘要: {old_summary or '无'}\n"
        f"已确认事实: {facts or ['无']}\n"
        f"待压缩旧轮次: \n{material}"
    )
    return [
        SystemMessage(content="你是上下文压缩器，只输出简洁中文摘要。"),
        HumanMessage(content=prompt),
    ]

def _fallback_summary(
    self,
    old_summary: str,
    older: list[ContextEntry],
) -> str:
    material = "; ".join(
        f"用户询问{item.user_text}，助手回答{item.assistant_text}"
        for item in older
    )
    combined = f"{old_summary}; {material}".strip("; ")
    max_chars = int(self._summary_max_tokens * self._chars_per_token)
    return combined[-max_chars:]

```

### 消息装配顺序不要随意变化

固定为角色提示词 → 摘要 → 已确认事实 → 近期原始轮次 → 当前问题。请求体 history 继续接收，但任何地方都不要把它加入 model\_messages。

## 第 7 步：扩展 AgentState

State 只存一轮图运行需要的数据；客户端连接对象仍放在闭包和 AgentRuntime 中，不能塞进 State。

**替换文件内容** agent\_service/src/agent\_service/graph/state.py

```

"""LangGraph 在单次运行中传递的状态结构。"""

from typing import TypedDict

from langchain_core.messages import BaseMessage

from agent_service.schemas.context import ContextSnapshot


class AgentState(TypedDict):
    """加入上下文后的一轮运行状态。"""

    request_id: int
    session_id: int
    user_id: int
    message: str
    model_route: str
    max_output_tokens: int
    role_name: str
    system_prompt: str

    # load_context 写入当前持久化快照和原始 revision。
    context_snapshot: ContextSnapshot
    context_revision: int

    # compact_context 按固定顺序装配, generate 只使用这个字段。
    model_messages: list[BaseMessage]
    final_answer: str

```

**第 8 步：把上下文节点接入现有 LangGraph**

这是核心改动。generate 不再自己创建 SystemMessage + HumanMessage，而是只消费 compact\_context 生成的 model\_messages。

**替换核心编排** agent\_service/src/agent\_service/graph/workflow.py

```

"""加入上下文后的 LangGraph 编排流程。"""

from collections.abc import Callable
from typing import Any

from langgraph.graph import END, START, StateGraph
from langgraph.graph.state import CompiledStateGraph
from langgraph.types import StreamWriter

from agent_service.core.cancellation import CancellationRegistry
from agent_service.core.exceptions import AgentCancelledError, EmptyModelOutputError
from agent_service.graph.state import AgentState

```

```

from agent_service.models.gateway import ModelGateway
from agent_service.services.context_manager import ContextManager
from agent_service.services.context_repository import ContextRepository

def build_agent_graph(
    model_gateway: ModelGateway,
    cancellation_registry: CancellationRegistry,
    context_repository: ContextRepository,
    context_manager: ContextManager,
) -> CompiledStateGraph:
    """编译 input → load → compact → generate → validate → persist。"""

    async def input_guard(
        state: AgentState,
        writer: StreamWriter,
    ) -> dict[str, str]:
        """保留现有输入规范化逻辑。"""

        # StreamWriter 是同步写入器，因此这里不加 await。
        writer(
            {
                "event": "status",
                "payload": {"stage": "safety", "message": "正在检查输入内容"},
            }
        )
        return {"message": " ".join(state["message"].split())}

    async def load_context(
        state: AgentState,
        writer: StreamWriter,
    ) -> dict[str, Any]:
        """按 userId + sessionId 加载上下文，不读取 MySQL 或 history。"""

        writer(
            {
                "event": "status",
                "payload": {"stage": "context", "message": "正在加载会话上下文"},
            }
        )
        snapshot = await context_repository.load(
            user_id=state["user_id"],
            session_id=state["session_id"],
        )
        return {
            "context_snapshot": snapshot,
            "context_revision": snapshot.revision,
        }

    async def compact_context(
        state: AgentState,
        writer: StreamWriter,
    ) -> dict[str, Any]:
        """超过软阈值时先压缩，再生成最终模型消息。"""

        if context_manager.needs_compaction(
            snapshot=state["context_snapshot"],
            system_prompt=state["system_prompt"],
            current_message=state["message"],
        ):
            writer(

```



```

        {
            "event": "status",
            "payload": {
                "stage": "context",
                "message": "上下文较长, 正在压缩历史信息",
            },
        }
    )

    snapshot, messages = await context_manager.prepare(
        snapshot=state["context_snapshot"],
        system_prompt=state["system_prompt"],
        current_message=state["message"],
        model_gateway=model_gateway,
    )
    return {"context_snapshot": snapshot, "model_messages": messages}

async def generate(
    state: AgentState,
    writer: StreamWriter,
) -> dict[str, str]:
    """使用 compact_context 已经装配好的消息流式生成。"""

    writer(
        {
            "event": "status",
            "payload": {"stage": "generation", "message": "正在生成回答"},
        }
    )
    answer_parts: list[str] = []
    async for text in model_gateway.stream(
        state["model_messages"],
        max_output_tokens=state["max_output_tokens"],
    ):
        if await cancellation_registry.is_cancelled(state["request_id"]):
            raise AgentCancelledError(f"requestId={state['request_id']} 已取消")
        answer_parts.append(text)
        writer({"event": "delta", "payload": {"content": text}})

    answer = "".join(answer_parts).strip()
    if not answer:
        raise EmptyModelOutputError("模型没有生成可展示内容")
    return {"final_answer": answer}

async def output_validate(
    state: AgentState,
    writer: StreamWriter,
) -> dict[str, str]:
    """只有校验通过的完整回答才允许进入 persist_context。"""

    del writer
    if not state["final_answer"].strip():
        raise EmptyModelOutputError("回答校验失败: 内容为空")
    return {}

async def persist_context(
    state: AgentState,
    writer: StreamWriter,
) -> dict[str, Any]:
    """保存完整一轮; 取消或生成异常不会执行到这里。"""

```

```

    if await cancellation_registry.is_cancelled(state["request_id"]):
        raise AgentCancelledError(f"requestId={state['request_id']} 已取消")

    writer(
        {
            "event": "status",
            "payload": {"stage": "context", "message": "正在保存会话上下文"},
        }
    )
    completed = context_manager.append_completed_turn(
        snapshot=state["context_snapshot"],
        request_id=state["request_id"],
        user_text=state["message"],
        assistant_text=state["final_answer"],
    )
    saved = await context_repository.save(
        completed,
        expected_revision=state["context_revision"],
    )
    return {
        "context_snapshot": saved,
        "context_revision": saved.revision,
    }

graph = StateGraph(AgentState)
graph.add_node("input_guard", input_guard)
graph.add_node("load_context", load_context)
graph.add_node("compact_context", compact_context)
graph.add_node("generate", generate)
graph.add_node("output_validate", output_validate)
graph.add_node("persist_context", persist_context)
graph.add_edge(START, "input_guard")
graph.add_edge("input_guard", "load_context")
graph.add_edge("load_context", "compact_context")
graph.add_edge("compact_context", "generate")
graph.add_edge("generate", "output_validate")
graph.add_edge("output_validate", "persist_context")
graph.add_edge("persist_context", END)
return graph.compile()

```

```
AgentGraphFactory = Callable[..., CompiledStateGraph]
```

### 取消为什么不会保存半截回答？

`persist_context` 位于 `output_validate` 之后。模型流中取消会直接抛 `AgentCancelledError`，图不会走到保存节点；保存节点开始前还会再检查一次取消标记。

## 第 9 步：修改 AgentRuntime 注入依赖

AgentRuntime 已经负责组装模型、取消注册表和图，因此 Repository 与 ContextManager 也从这里注入最自然。

**修改文件** agent\_service/src/agent\_service/services/agent\_runtime.py

```
# 文件: src/agent_service/services/agent_runtime.py
# 操作: 修改构造函数、build_agent_graph 调用和初始 state。

class AgentRuntime:
    """协调运行注册、LangGraph 流消费和资源释放。"""

    def __init__(
        self,
        model_gateway: ModelGateway,
        role_profile_provider: RoleProfileProvider,
        context_repository: ContextRepository,
        context_manager: ContextManager,
        cancellation_registry: CancellationRegistry | None = None,
    ) -> None:
        self.model_gateway = model_gateway
        self.role_profile_provider = role_profile_provider
        self.cancellation_registry = cancellation_registry or CancellationRegistry()
        self.graph = build_agent_graph(
            model_gateway,
            self.cancellation_registry,
            context_repository,
            context_manager,
        )

    # load_active_role、cancel 和 stream 的事件消费逻辑保持不变。

    async def stream(self, request, role_profile):
        await self.cancellation_registry.register(request.request_id)
        state = {
            "request_id": request.request_id,
            "session_id": request.session_id,
            "user_id": request.user_id,
            "message": request.message,
            "model_route": request.policy.model_route,
            "max_output_tokens": request.policy.max_output_tokens,
            "role_name": role_profile.name,
            "system_prompt": role_profile.system_prompt,
            # 这三个字段会在 load_context / compact_context 中被覆盖。
            "context_snapshot": ContextSnapshot.empty(
                user_id=request.user_id,
                session_id=request.session_id,
            ),
            "context_revision": 0,
            "model_messages": [],
            "final_answer": "",
        }

        try:
            async for part in self.graph.astream(
                state,
                stream_mode="custom",
                version="v2",
            ):
                if await self.cancellation_registry.is_cancelled(request.request_id):
                    raise AgentCancelledError(
                        f"requestId={request.request_id} 已取消"
                    )
                if part["type"] != "custom":
                    continue
                yield AgentEvent.model_validate(part["data"])
```

```
finally:
    await self.cancellation_registry.finish(request.request_id)
```

## 第 10 步：用 FastAPI lifespan 管理连接

Redis 和 AsyncMongoClient 都应该每个进程创建一次，而不是每条聊天创建一次。应用退出时显式关闭，pytest 则注入内存 Repository。

**替换文件内容** agent\_service/src/agent\_service/main.py

```
"""FastAPI 应用工厂和 Redis/MongoDB 生命周期。"""

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI
from pymongo import AsyncMongoClient
from redis.asyncio import Redis
from redis.exceptions import RedisError

from agent_service import __version__
from agent_service.api.routes import chat, health
from agent_service.config import Settings, get_settings
from agent_service.models.gateway import ModelGateway, create_model_gateway
from agent_service.services.agent_runtime import AgentRuntime
from agent_service.services.context_manager import ContextManager
from agent_service.services.context_repository import (
    ContextRepository,
    RedisMongoContextRepository,
)
from agent_service.services.role_profile import RoleProfileProvider

logger = logging.getLogger(__name__)

def create_app(
    *,
    settings: Settings | None = None,
    model_gateway: ModelGateway | None = None,
    context_repository: ContextRepository | None = None,
) -> FastAPI:
    """创建应用；测试可注入内存 Repository，避免访问真实数据库。"""

    resolved_settings = settings or get_settings()
    resolved_gateway = model_gateway or create_model_gateway(resolved_settings)

    @asynccontextmanager
    async def lifespan(app: FastAPI):
        """每个进程只创建一套异步客户端，并在进程退出时关闭。"""

        redis_client: Redis | None = None
        mongo_client: AsyncMongoClient | None = None
```

```

if context_repository is not None:
    # pytest 传入 InMemoryContextRepository 时不连接开发环境。
    repository = context_repository
else:
    redis_password = resolved_settings.redis_password
    redis_client = Redis(
        host=resolved_settings.redis_host,
        port=resolved_settings.redis_port,
        password=(
            redis_password.get_secret_value()
            if redis_password is not None
            else None
        ),
        db=resolved_settings.redis_database,
        decode_responses=True,
        socket_connect_timeout=(
            resolved_settings.redis_connect_timeout_seconds
        ),
        socket_timeout=resolved_settings.redis_socket_timeout_seconds,
    )

    mongo_client = AsyncMongoClient(
        resolved_settings.mongodb_uri.get_secret_value(),
        serverSelectionTimeoutMS=(
            resolved_settings.mongodb_server_selection_timeout_ms
        ),
        connectTimeoutMS=resolved_settings.mongodb_connect_timeout_ms,
        tz_aware=True,
    )

    # MongoDB 是必要依赖：启动探测失败时让服务启动失败。
    await mongo_client.admin.command("ping")

    # Redis 是可降级缓存：探测失败只记录告警，聊天仍可回源 MongoDB。
    try:
        await redis_client.ping()
    except RedisError:
        logger.warning("Redis 启动探测失败，将使用 MongoDB 回源", exc_info=True)

    repository = RedisMongoContextRepository(
        redis_client=redis_client,
        mongo_client=mongo_client,
        mongo_database=resolved_settings.mongodb_database,
        redis_ttl_seconds=resolved_settings.context_redis_ttl_seconds,
    )

app.state.agent_runtime = AgentRuntime(
    resolved_gateway,
    RoleProfileProvider(resolved_settings.role_config_path),
    repository,
    ContextManager(resolved_settings),
)

try:
    yield
finally:
    # redis-py 和 PyMongo Async 都需要显式异步关闭。
    if redis_client is not None:
        await redis_client.aclose()
    if mongo_client is not None:
        await mongo_client.close()

```

```

logging.basicConfig(
    level=getattr(logging, resolved_settings.log_level.upper(), logging.INFO),
    format="%asctime)s %(levelname)s %(name)s %(message)s",
)
app = FastAPI(
    title="XinChuang Agent Service",
    version=__version__,
    description="信创智能客服独立 LangChain/LangGraph 智能体服务",
    lifespan=lifespan,
)
app.state.settings = resolved_settings
api_prefix = "/internal/ai/v1"
app.include_router(health.router, prefix=api_prefix)
app.include_router(chat.router, prefix=api_prefix)
return app

```

```
app = create_app()
```

### 多 worker 时的含义

每个 uvicorn worker 是独立进程，因此会各自创建 Redis 连接池和 AsyncMongoClient；不要在父进程创建客户端后跨进程共享。

## 第 11 步：把上下文异常映射为 SSE error

SSE 响应建立以后不能再修改 HTTP 状态码，所以沿用现有路由风格，通过终态 error 告诉 Java。

**修改文件** agent\_service/src/agent\_service/api/routes/chat.py

```

# 文件: src/agent_service/api/routes/chat.py
# 操作 1: 补充异常导入。
from agent_service.core.exceptions import (
    ContextRevisionConflictError,
    ContextStoreUnavailableError,
    ContextTooLargeError,
)

# 操作 2: 把下面分支放到 except Exception 之前。
except ContextStoreUnavailableError:
    yield frame(
        "error",
        {
            "code": "CONTEXT_STORE_UNAVAILABLE",
            "message": "会话上下文服务暂时不可用",
            "retryable": True,
        },
    )
except ContextRevisionConflictError:
    yield frame(

```

```

        "error",
        {
            "code": "CONTEXT_REVISION_CONFLICT",
            "message": "会话正在被其他请求更新, 请重试",
            "retryable": True,
        },
    )
except ContextTooLargeError:
    yield frame(
        "error",
        {
            "code": "INPUT_TOO_LARGE",
            "message": "当前问题和必要上下文过长, 请缩短后重试",
            "retryable": False,
        },
    )
)

```

## 第 12 步：修改测试夹具并增加上下文测试

单元测试必须离线运行。不要为了测试 load\_context 就连接虚拟机数据库，否则测试会变慢、互相污染，也无法稳定复现错误。

**修改文件** agent\_service/tests/conftest.py

```

# 文件: tests/conftest.py
# 操作: 导入内存仓储, 并在 create_app 中注入。

from agent_service.services.context_repository import InMemoryContextRepository

@pytest.fixture
def context_repository() -> InMemoryContextRepository:
    """每个测试使用独立内存仓储, 避免测试之间互相污染。"""

    return InMemoryContextRepository()

@pytest.fixture
def client(context_repository: InMemoryContextRepository) -> TestClient:
    """测试客户端不会连接真实 Redis、MongoDB 或模型。"""

    settings = Settings(
        environment="test",
        model_provider="mock",
        model_name="mock-model",
        internal_auth_enabled=False,
        # Settings 新增了必填 Mongo URI; 测试只需合法占位值。
        mongodb_uri="mongodb://unused",
    )
    app = create_app(
        settings=settings,
        model_gateway=MockModelGateway(),
    )

```

```

        context_repository=context_repository,
    )
    with TestClient(app) as test_client:
        yield test_client

```

### 新增文件 agent\_service/tests/test\_context\_system.py

```

# 新增文件: tests/test_context_system.py

import json

from fastapi.testclient import TestClient

from agent_service.services.context_repository import InMemoryContextRepository

def _events(response) -> list[dict[str, object]]:
    """把 SSE data 行转换为字典, 方便断言。"""

    return [
        json.loads(line.removeprefix("data: "))
        for line in response.iter_lines()
        if line.startswith("data: ")
    ]

async def test_completed_chat_is_saved_as_model_context(
    client: TestClient,
    chat_request: dict[str, object],
    context_repository: InMemoryContextRepository,
) -> None:
    """正常完成后 revision 增加, 并保存一轮上下文。"""

    with client.stream(
        "POST",
        "/internal/ai/v1/chat/stream",
        json=chat_request,
    ) as response:
        events = _events(response)

    snapshot = await context_repository.load(user_id=7, session_id=3)
    assert events[-1]["event"] == "done"
    assert snapshot.revision == 1
    assert snapshot.last_request_id == 21
    assert len(snapshot.recent_entries) == 1
    assert snapshot.recent_entries[0].user_text == "UOS 打印机无法识别怎么办? "

async def test_second_chat_reuses_same_session_context(
    client: TestClient,
    chat_request: dict[str, object],
    context_repository: InMemoryContextRepository,
) -> None:
    """同一会话第二轮完成后 revision 应从 1 增加到 2。"""

    for request_id, message in [(21, "第一轮问题"), (22, "第二轮追问")]:
        chat_request["requestId"] = request_id

```



```

chat_request["message"] = message
with client.stream(
    "POST",
    "/internal/ai/v1/chat/stream",
    json=chat_request,
) as response:
    assert _events(response)[-1]["event"] == "done"

snapshot = await context_repository.load(user_id=7, session_id=3)
assert snapshot.revision == 2
assert [item.request_id for item in snapshot.recent_entries] == [21, 22]

```

## 第 13 步：按顺序运行和验证

### 执行命令 agent\_service/

```

# 1. 让 uv 根据新增依赖同步环境。
uv sync --dev

# 2. 统一格式并做静态检查。
uv run ruff format .
uv run ruff check .

# 3. 测试必须使用 InMemoryContextRepository, 不能依赖开发数据库。
uv run pytest

# 4. 启动真实服务; 此时 lifespan 会探测 MongoDB 和 Redis。
uv run agent-service

```

服务启动成功后，连续向同一个 sessionId 发送两条不同 requestId 的聊天请求。第二轮应先出现“正在加载会话上下文”，回答完成后 MongoDB revision 变为 2。

### 检查 Redis DB 4

```

# 在能够访问虚拟机 Redis 的机器执行; 确认 Agent 只写 database 4。
redis-cli -h 192.168.100.128 -p 6379 -a <Redis 密码> -n 4 \
    SCAN 0 MATCH 'xc:agent:context:v1:*'

# 查看某个会话剩余 TTL, 应接近 604800 秒, 并在成功读取后重新刷新。
redis-cli -h 192.168.100.128 -p 6379 -a <Redis 密码> -n 4 \
    TTL 'xc:agent:context:v1:session:<userId>:<sessionId>'

```

## 检查 MongoDB

```
# 进入现有 mongo 容器。
docker exec -it mongo mongosh \
  --username mongo \
  --password mongo \
  --authenticationDatabase admin

# 在 mongosh 中检查 Agent 上下文。
use xinchuang_agent_context
db.context_sessions.find().pretty()
db.context_turns.find().sort({ createdAt: -1 }).limit(5).pretty()
```

## 遇到问题时先按这里排查

| 现象                 | 最可能原因                       | 先检查                             |
|--------------------|-----------------------------|---------------------------------|
| 启动时 Mongo 认证失败     | root 用户却使用了目标库 authSource   | URI 改为 authSource=admin         |
| Redis 没有 Key 但聊天正常 | Redis 写入降级，或查看了 DB 3        | 确认 -n 4 和日志告警                   |
| 第二轮仍像首次会话          | userId/sessionId 变化或保存节点未执行 | Mongo context_sessions 和 SSE 终态 |
| 重复请求 revision 增加   | lastRequestId 没有正确保存/校验     | 检查 requestId 和 Mongo 文档字段       |
| pytest 尝试连接虚拟机     | create_app 未注入内存 Repository | 检查 conftest.py                  |
| 压缩后仍超长             | 当前问题或系统提示本身过大               | 返回 INPUT_TOO_LARGE，不继续裁剪当前问题    |

## 完成标准

- 第一轮聊天结束后，MongoDB 出现一个 context\_sessions 文档和一个 context\_turns 文档。

- 第二轮会话会加载第一轮上下文，revision 从 1 增加到 2。
- Redis DB 4 出现带 xc:agent:context:v1: 前缀的 Key，TTL 接近七天。
- 停止 Redis 后仍能从 MongoDB 读取并继续聊天；日志显示缓存降级。
- 停止 MongoDB 后在正式模型生成前返回 CONTEXT\_STORE\_UNAVAILABLE。
- history=null、history=[] 和缺省 history 都不会进入 model\_messages。
- 取消或模型空输出不会保存一条完整上下文。
- uv run ruff check . 和 uv run pytest 全部通过。

## 第一版先不要继续增加的东西

先把上面的闭环跑通，再考虑 RAG、工具、清理接口、LangGraph checkpoint 或更精确 tokenizer。它们都可以沿用同一个 ContextRepository 和 model\_messages 边界继续扩展，不需要现在一起实现。

## 参考接口

- redis-py asyncio：异步命令需要 await，应用退出时显式 await Redis.aclose()。  
[https://redis.readthedocs.io/en/stable/examples/asyncio\\_examples.html](https://redis.readthedocs.io/en/stable/examples/asyncio_examples.html)
- PyMongo Async：FastAPI 可使用 AsyncMongoClient，网络操作需要 await。  
<https://www.mongodb.com/docs/languages/python/pymongo-driver/current/reference/migration/>
- LangGraph 自定义流：异步节点可接收 StreamWriter，writer(...) 本身不加 await。  
<https://langchain-ai.github.io/langgraph/cloud/concepts/streaming/>